Chapter 12

The (untyped) lambda calculus

The lambda calculus was invented by Alonzo Church around 1930 as an attempt to formalize a notation for computable functions. It is important to have some basic familiarity with the lambda calculus because it forms the foundation for the definition of functional programming languages such as Haskell.

In the conventional set theoretic formulation of mathematics, a function f can be represented by its graph—that is, a binary relation R_f between the sets $domain(f)$ and $codomain(f)$ consisting of all pairs $(x, f(x))$. Not all binary relations represent functions—functions are single-valued, so if $(x, y) \in R_f$ and $(x, y') \in R_f$ then we must have $y = y'$. In mathematics, functions are almost always total, so we also have an additional requirement that for each x in $domain(f)$, there exists a pair $(x, y) \in R_f$. Representing a function in terms of its graph is called an *extensional* definition—two functions are equal if and only if their graphs are equal. This is analogous to the definition of equality in set theory—two sets are equal if and only if they have the same elements.

From a computational viewpoint, it is not sensible to represent functions merely in terms of graphs. Clearly, all sorting algorithms define functions that have the same graph. However, from a computational viewpoint, we would like to distinguish efficient sorting algorithms from inefficient ones.

What we need is an *intensional* representation of functions—a representation that captures not just the output value for each input, but also how this value is arrived at.

With this background, we plunge into the (pure untyped) lambda calculus.

12.1 Syntax

The set of lambda expressions is generated by what we would, in modern day computer science terminology, call a context-free grammar.

Let *Var* be a countably infinite set of variables. We define the set Λ of lambda expressions as follows:

$$\Lambda = x \mid \lambda x.M \mid MM'$$

where $x \in Var$ and $M, M' \in \Lambda$.

Thus, the syntax of lambda expressions contains two ways to generate new expressions from old ones. The construction $\lambda x.M$ is called *abstraction*, while the construction MM' is called *application*.

The intention is that $\lambda x.M$ is an expression representing a function of x whose body (or rule for computation) is given by M. Thus, $\lambda x.M$ "abstracts" the computation rule described in M to operate on arbitrary input values x. This is analogous to writing $f(x) = M$ without having to assign an unnecessary "name" f to the function (after all, $f(x) = M$ and $g(x) = M$ are clearly the same function, so the names f and g carry no significance.)

The expression MM' is intended to represent the action of "applying" the function encoded by M to the argument M'.

12.2 Life without types

At this point, we should remind ourselves that these intended meanings do not stop us from constructing seemingly nonsensical expressions such as xx.

At first sight, it seems strange that we should try to embark on a syntax for functions in which there is no notion of "type". However, untyped formalisms are familiar to us

- Set theory is untyped. Everything is a set. To do arithmetic, we define certain sets to denote numbers. Functions on numbers apply in principle to all sets but are "meaningless" in the wrong context.
- A computer's memory is intrinsically untyped. Consider the operation of *dereferencing a pointer* in C—that is, treat a memory location as a pointer and look up the location that it points to. This operation is performed without checking if the contents of the pointer are of the right "type"—that is, a memory location legally available to this program. If the contents of the pointer are valid, the operation succeeds. However, if the pointer is invalid (for instance, if it is uninitialized), the operation fails (sometimes dramatically, as we all know!)

The point is that we have a syntax for representing data (sets, bit strings) and we have rules for manipulating these representations. Some representations are more meaningful than others, but the manipulation rules can be applied uniformly to both the meaningful and the meaningless representations. All we ask is that if the original data represents a meaningful value, then, after manipulation, it still represents a meaningful value.

A similar view is taken in the lambda calculus—for instance MM' represents application if the structure of the expression M corresponds to a "function". Otherwise, we don't care what this construction means (e.g., if MM' is xx, as describe earlier).

12.3 The rule β

The basic rule for computing with the lambda calculus is called β and is given by:

$$(\lambda x.M)M' \rightarrow_\beta M\{x \leftarrow M'\}$$

where $M\{x \leftarrow M'\}$ represents the effect of substituting all *free* occurrences of x in M uniformly by M'. (We shall formally define what we mean by a free occurrence of x shortly.) The β rule is one that we use unconciously all the time: If $f(x) = 2x^2 + 3x + 4$, then $f(7) = 2 \cdot 7^2 + 3 \cdot 7 + 4$ which is just $2x^2 + 3x + 4\{x \leftarrow 7\}$.

From the β rule it is clear that an application MM' is meaningful only if M is of the form $\lambda x.M$". Thus, though we can write terms like xx, we cannot do much with them.

Effectively, the β rule is the *only* rule we need to set up the lambda calculus. For the moment, think of it as a rewriting rule for expressions that can be applied to any subterm in a lambda expression. We will formalize this later.

12.4 Variable capture

An important issue to keep in mind when applying the rule β is that the substitution for x should not "capture" variables. Consider, for instance, the application $(\lambda x.(\lambda y.xy))y$. Using the β rule naively, we get $(\lambda x.(\lambda y.xy))y \rightarrow_\beta \lambda y.yy$ However, the y that comes in to $\lambda y.xy$ by the substitution for x is a *new* y and should not be confused with the y that is used to define the inner function. Thus, we should redefine the inner function with a fresh variable before making the substitution, as follows:

$$(\lambda x.(\lambda y.xy))y = (\lambda x.(\lambda z.xz))y \rightarrow_\beta \lambda z.yz$$

Clearly, changing the name of the variable defining a function does not affect the definition—there is no difference between the functions $f(x) = 2x + 5$ and $f(z) = 2z + 5$.

To formalize this notion we have to define *free* and *bound* variables. Intuitively, a variable is free in an expression until it is "bound" by an abstraction. Thus, in the expression xy, both x and y are free. If we abstract y as $\lambda y.xy$, x remains free but y becomes bound. More formally, we can define two sets $FV(M)$ (*free variables of* M) and $BV(M)$ (*bound variables of* M) inductively, as follows:

- $FV(x) = \{x\}$, for any variable x
- $FV(\lambda x.M) = FV(M) - \{x\}$
- $FV(MM') = FV(M) \cup FV(M')$
- $BV(x) = \emptyset$, for any variable x
- $BV(\lambda x.M) = BV(M) \cup \{x\}$
- $BV(MM') = BV(M) \cup BV(M')$

When we apply the β rule to transform $\lambda x.MM'$ into $M\{x \leftarrow M'\}$, we need to check that no free variable in M' clashes with a bound variable in M. In the example above,

$$(\lambda x.(\lambda y.xy))y \rightarrow_\beta \lambda y.yy,$$

y is bound in the expression $M = \lambda y.xy$, while y is free in the expression $M' = y$.

In the literature, several approaches have been proposed to deal with the variable renaming problem that is required for the β rule to work properly. For simplicity, we shall just assume that we always rename the bound variables in M to avoid "capturing" free variables from M'.

12.5 Encoding arithmetic

Having set up this rather abstract notation for computable functions, let us see an example of how to use it.

Before proceeding, we set up some conventions. Implicitly, lambda expressions associate to the left. Thus, the expresssion $M_1M_2M_3$ denotes $((M_1M_2)M_3)$. Also, if we have an abstraction with multiple variables at the outermost level, such as $\lambda x.(\lambda y.xy)$, we can pull out all the variables at the outermost level into a single sequence such as $\lambda xy.xy$.

In set theory, the number n is identified, in a sense, with a set that has nesting level of depth n. For instance, we start with the emptyset for representing the number 0 and, inductively, use the set $\{0, 1, .., n-1\}$ to represent the number n. Thus we have

$$\begin{array}{rcl} 0 & = & \emptyset \\ 1 & = & \{\emptyset\} \\ 2 & = & \{\emptyset, \{\emptyset\}\} \\ 3 & = & \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\} \\ \ldots & & \end{array}$$

In the lambda calculus, we encode the number n by the number of times we apply a function to an argument. Thus, the number n is a lambda expression that takes two arguments and applies the first argument to the second argument n times. Let $\langle n \rangle$ be an abbreviation for the lambda term denoting n. Then, we have:

$$\begin{array}{rcl} \langle 0 \rangle & = & \lambda fx.x \\ \langle n+1 \rangle & = & \lambda fx.f(\langle n \rangle fx) \end{array}$$

This encoding was proposed by Church and terms of the form $\langle n \rangle$ are hence called the *Church numerals*. Concretely, we have

$$\langle 1 \rangle = \lambda fx.f(\langle 0 \rangle fx) = \lambda fx.(f((\lambda fx.x)fx))$$

Note that $\langle 0 \rangle gy \rightarrow_\beta (\lambda x.x)y \rightarrow_\beta y$. Thus $\langle 0 \rangle fx \rightarrow_\beta x$. So, inside the definition of $\langle 1 \rangle$, we get

$$\langle 1 \rangle = \ldots = \lambda fx.(f \underbrace{\lambda fx.x)fx)}_{\text{apply } \beta}) \rightarrow_\beta \lambda fx.(fx)$$

Since $\langle 1 \rangle gy \rightarrow_\beta (\lambda x.(gx))y \rightarrow_\beta gy$, we have

$$\langle 2\rangle = \lambda fx.f(\langle 1\rangle fx) = \lambda fx.(f(\underbrace{\lambda fx.(fx)fx)}_{\text{apply }\beta}) \rightarrow_\beta \lambda fx.(f(fx))$$

and

$$\langle 2\rangle gy \rightarrow_\beta \lambda x.(g(gx))y = g(gy)$$

Let us write $g^k y$ to denote the term $g(g(\ldots(gy)))$ with k applications of g to y. Then, it is not difficult to show inductively that

$$\langle n\rangle = \lambda fx.f(\langle n{-}1\rangle fx) \rightarrow_\beta \ldots \rightarrow_\beta \lambda fx.(f^n x)$$

We can now define arithmetic functions such as *successor*, *plus*, ... For instance, *successor* is just $\lambda pfx.f(pfx)$. To see how this works, we observe that

$$(\lambda pfx.f(pfx))\langle n\rangle \rightarrow_\beta \lambda fx.f(\langle n\rangle fx) \rightarrow_\beta \lambda fx.f(f^n x) = \lambda fx.f^{n+1}x = \langle n{+}1\rangle$$

Thus, if the function representing successor is applied a term of the right "type", it yields the expected answer. We could also apply it to a "meaningless" term, in which case we get a "meaningless" answer!

Here is a definition for *plus*: $\lambda pqfx.pf(qfx)$. We can verify that

$$\begin{array}{rcl}
(\lambda pqfx.pf(qfx))\langle m\rangle\langle n\rangle & \rightarrow_\beta & (\lambda qfx.\langle m\rangle f(qfx))\langle n\rangle \\
& \rightarrow_\beta & (\lambda fx.\langle m\rangle f(\langle n\rangle fx)) \\
& \rightarrow_\beta & (\lambda fx.\langle m\rangle f(f^n x)) \\
& \rightarrow_\beta & (\lambda fx.f^m(f^n x)) \\
& = & (\lambda fx.f^{m+n}x) = \langle m{+}n\rangle
\end{array}$$

Similarly, we define *multiplication* and *exponentiation* as follows:

$$\begin{array}{rcl}
\textit{multiplication} & : & \lambda pqfx.q(pf)x \\
\textit{exponentiation} & : & \lambda pq.(pq)
\end{array}$$

12.6 One step reduction

Recall that we introduced the β rule

$$(\lambda x.M)M' \rightarrow_\beta M\{x \leftarrow M'\}$$

and said, informally, that we would permit this rule to be used in all "contexts". The β rule is not the only basic rule possible. For instance, observe that the two terms $\lambda x.(Mx)$ and M are equivalent with respect to β reduction—for any term M', $(\lambda x.(Mx))M' \rightarrow_\beta MM'$. This can be formalized by a rule (normally called η) which says

$$\lambda x.(Mx) \rightarrow_\eta M$$

Given a set of *basic* rules such as β, η, ..., we can inductively define a one step reduction that permits any of these basic rules to be used in any context. Let us denote one step reduction by $\rightarrow$. We define $\rightarrow$ through inference rules such as the following:

$$\text{if } M \rightarrow_x M' \text{ for some basic rule } x \text{ such as } \beta, \eta, \ldots, \text{ then } M \rightarrow M'$$

Following the conventional notation used in logic, we present such a rule in the following form

$$\frac{M \rightarrow_x M'}{M \rightarrow M'}$$

Here is the complete set of rules defining $\rightarrow$:

$$\frac{M \rightarrow_x M''}{M \rightarrow M} \quad \frac{M \rightarrow M'}{\lambda x.M \rightarrow \lambda x.M'} \quad \frac{M \rightarrow M'}{MN \rightarrow M'N} \quad \frac{N \rightarrow N'}{MN \rightarrow MN'}$$

Notice that all that we have done is to formalize the fact that we permit the basic reductions $\rightarrow_x$ within any subterm. In the "calculations" we have seen so far, we have already informally used this form of applying the β rule.

In the discussion that follows, we shall dispense with the η rule and assume that the only basic rule is the rule β.

12.7 Normal forms

As we had discussed in the context of Haskell, we are interested in terms that cannot be reduced. These denote "values" and are called *normal forms*.

Formally, we are interested in properties of the relation $\rightarrow^*$, the reflexive and transitive closure of $\rightarrow$. We have already anticipated some of the questions we would like to ask about $\rightarrow^*$ in our discussion of Haskell.

- Does every term reduce to a normal form?
- Can a term reduce to more than one normal form, depending on the order in which we perform reductions?
- If a term has a normal form, can we always find it?

Consider the term $(\lambda x.xx)(\lambda x.xx)$. Only one reduction step is possible and this makes two copies of the argument and returns us to the original term $(\lambda x.xx)(\lambda x.xx)$. The reduction process for this term never terminates, so this is an example of a lambda term without a normal form. In the literature, the term $(\lambda x.xx)(\lambda x.xx)$ is usually denoted Ω.

Consider now the term $(\lambda yz.z)$ that ignores its first argument. If we apply $(\lambda yz.z)$ to Ω, we have two possible reductions.

- $(\lambda yz.z)((\lambda x.xx)(\lambda x.xx)) \rightarrow \lambda z.z$, using outermost reduction.

- $(\lambda yz.z)((\lambda x.xx)(\lambda x.xx)) \rightarrow (\lambda yz.z)((\lambda x.xx)(\lambda x.xx))$, using innermost reduction.

Clearly, if we keep making the second choice, we will never reach a normal form, even though it is always possible to reach a normal form in one step by choosing the first reduction.

To answer the question of multiple normal forms, we will state a more general result shortly.

First, we define a notion of equivalence based on $\rightarrow^*$.

12.8 $\rightarrow^*$-equivalence

We define an equivalence on lambda terms, which we shall write $\leftrightarrow$, based on the many step reduction relation $\rightarrow^*$. Basically, $\leftrightarrow$ is the symmetric transitive closure of $\rightarrow^*$. It can be defined inductively using inference rules in the same way that we defined $\rightarrow$ based on $\rightarrow_x$:

$$\frac{M \rightarrow^* N}{M \leftrightarrow N} \quad \frac{M \leftrightarrow N}{N \leftrightarrow M} \quad \frac{M \leftrightarrow N, N \leftrightarrow P}{M \leftrightarrow P}$$

We can do the same with any reflexive, transitive relation R— the symmetric, transitive closure of R defines an equivalence relation $\overset{R}{\leftrightarrow}$.

12.9 Church-Rosser property

Let R be a binary relation that is reflexive and transitive. We say that R has the *diamond property* if, whenever $X\ R\ Y$ and $X\ R\ Z$, then there is a W such that $Y\ R\ W$ and $Z\ R\ W$. If a relation R has the diamond property, we say that R is Church-Rosser.

Theorem *(Church-Rosser)* Let R be Church-Rosser. Then $M \overset{R}{\leftrightarrow} N$ implies there exists Z, $M\ R\ Z$ and $N\ R\ Z$

Proof By induction on the definition of $\overset{R}{\leftrightarrow}$.

(i) $M \overset{R}{\leftrightarrow} N$ because $M\ R\ N$. Then chooze $Z \equiv N$.

(ii) $M \overset{R}{\leftrightarrow} N$ because $N \overset{R}{\leftrightarrow} M$. Trivial.

(iii) $M \overset{R}{\leftrightarrow} N$ because $M \overset{R}{\leftrightarrow} P$ and $P \overset{R}{\leftrightarrow} N$. By the induction hypothesis, we have Y such that $M\ R\ Y$ and $P\ R\ Y$ and Z such that $P\ R\ Z$ and $N\ R\ Z$.

Since $P\ R\ Y$, $P\ R\ Z$ and R is Church-Rosser, there is a term W such that $Y\ R\ W$ and $Z\ R\ W$.

But then $M\ R\ W$ and $N\ R\ W$.

□

We have the following useful Corollary:

Corollary Let R be Church-Rosser. Then a term can have at most one R-normal form (where an R-normal form for M is, as expected, a term N such that $M\ R^*\ N$ and there is no P such that $N\ R\ P$.)

Proof Suppose N_1 and N_2 are both normal forms for M. Then, $N_1 \overset{R}{\leftrightarrow} M \overset{R}{\leftrightarrow} N_2$, so $N_1 \overset{R}{\leftrightarrow} N_2$. There must then be a common Z such that $N_1\ R\ Z$ and $N_2\ R\ Z$. Since neither N_1 nor N_2 can be reduced, it follows that $N1 \equiv N_2 \equiv Z$. □

Recall that our goal is to check whether normal forms in the lambda calculus are unique. One way to do this is to show that $\rightarrow^*$ has the Church-Rosser property.

For any relation $\Rightarrow$, if $\Rightarrow$ has the Church-Rosser property, then so does $\Rightarrow^*$. The proof of this is easy to see pictorially.

$$\begin{array}{ccccccccccc}
M & \Rightarrow & & \Rightarrow & & \Rightarrow & & \Rightarrow & & \Rightarrow & N_1 \\
\Downarrow & & \Downarrow & & \Downarrow & & \Downarrow & & \Downarrow & & \Downarrow \\
 & \Rightarrow & & \Rightarrow & & \Rightarrow & & \Rightarrow & & \Rightarrow & \\
\Downarrow & & \Downarrow & & \Downarrow & & \Downarrow & & \Downarrow & & \vdots \\
 & \Rightarrow & & \Rightarrow & & \Rightarrow & & \Rightarrow & & \cdots & \\
\Downarrow & & \Downarrow & & \Downarrow & & \vdots & & & & \\
N_2 & \Rightarrow & & \Rightarrow & & \cdots & & & & &
\end{array}$$

If $M \Rightarrow^* N_1$ (the horizontal axis) and $M \Rightarrow^* N_2$ (along the vertical axis), we can break up both reductions into a sequence of individual steps. From the fact that $\Rightarrow$ is Church-Rosser, we can complete the diamond for each single step in the sequence. In the end, we get a complete rectangle between N_1 and N_2 and the corner opposite M is the common term that we can obtain from N_1 and N_2.

Unfortunately, $\rightarrow$ does not have the Church-Rosser property, so we cannot use this strategy to establish that $\rightarrow^*$ is Church-Rosser! Consider the term $(\lambda x.xx)((\lambda x.x)(\lambda x.x))$. We have two reductions possible:

- $(\lambda x.xx)((\lambda x.x)(\lambda x.x)) \rightarrow ((\lambda x.x)(\lambda x.x))((\lambda x.x)(\lambda x.x))$ (Outermost)
- $(\lambda x.xx)((\lambda x.x)(\lambda x.x)) \rightarrow ((\lambda x.xx)(\lambda x.x)$ (Innermost)

If we take the second option, we can then in one step get

$$(\lambda x.xx)(\lambda x.x) \rightarrow ((\lambda x.x)(\lambda x.x))$$

We can reach this term from the first option as well, but it requires *two* steps!

The solution lies in defining an alternative notion of one step reduction from the basic beta reduction $\rightarrow_\beta$ such that

(i) This new reduction is Church-Rosser.

(ii) Its reflexive, transitive closure is equal to $\rightarrow^*$.

We define this new reduction $\twoheadrightarrow$ as follows.

$$M \twoheadrightarrow M \quad \frac{M \twoheadrightarrow M'}{\lambda x.M \twoheadrightarrow \lambda x.M'} \quad \frac{M \twoheadrightarrow M', N \twoheadrightarrow N'}{MN \twoheadrightarrow M'N'} \quad \frac{M \twoheadrightarrow M', N \twoheadrightarrow N'}{(\lambda x.M)N \twoheadrightarrow M'\{x \leftarrow N'\}}$$

The last inference rule builds in β reduction. The crux of the relation $\twoheadrightarrow$ is that it combines nonoverlapping $\rightarrow$ reductions in one parallel step. For instance, if both M and N can be reduced to M' and N', respectively, then MN can be reduced to $M'N'$ in one step.

We claim, without proof, that $\twoheadrightarrow$ is Church-Rosser and that $\twoheadrightarrow^*$ is the same as $\rightarrow^*$. Assuming the claim, it then follows that $\rightarrow^*$ is Church-Rosser and so the normal form for a term in the lambda calculus is unique, if it exists.

12.10 Computability

We now argue that all computable functions can be described in the lambda calculus. As is usual in computability theory, we restrict our attention to functions on natural numbers. Recall that we can encode natural numbers in the lambda calculus via the Church numerals $\langle n \rangle \equiv \lambda f x.f^n x$. Our goal now is to show that for every computable function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ we can find a lambda term $\langle f \rangle$ that encodes f so that for every k-tuple $n_1, n_2, \ldots, n_k$

$$\langle f \rangle \langle n_1 \rangle \langle n_2 \rangle \ldots \langle n_k \rangle \rightarrow^* \langle f(n_1, n_2, \ldots, n_k) \rangle$$

In other words, the result of applying the encoding of f to the encodings of the arguments n_1, n_2, ..., n_k should be the encoding of the result $f(n_1, n_2, \ldots, n_k)$. Notice that on the left hand side we have used currying to break up the k-tuple of arguments to f into a sequence of k arguments to $\langle f \rangle$.

Recursive functions

One of the many formulations of computable functions that arose in the quest to define effective computability is that of *recursive functions*, due to Gödel. The recursive functions are defined inductively in terms of a set of initial functions and rules for combining functions, as follows.

Initial functions

- *Zero*: $Z(n) = 0$.
- *Successor*: $S(n) = n{+}1$.
- *Projection*: $\Pi_i^k(n_1, n_2, \ldots, n_k) = n_i$.[1]

[1]Notice that the projection functions are actually form an infinite family, one for each choice of k and $i \in \{1, 2, \ldots, k\}$.

Composition

Given $f : \mathbb{N}^k \to \mathbb{N}$ and $g_1, g_2, \ldots, g_k : \mathbb{N}^h \to \mathbb{N}$, the *composition* of f and $g_1, g_2, \ldots, g_k$, denoted $f \circ (g_1, g_2, \ldots, g_k)$, is the function

$$f \circ (g_1, g_2, \ldots, g_k)(n_1, n_2, \ldots, n_h) = \\ f(g_1(n_1, n_2, \ldots, n_h), g_2(n_1, n_2, \ldots, n_h), \ldots, g_k(n_1, n_2, \ldots, n_h))$$

For example, the function $f(n) = n{+}2$ can be defined as the composition $f = S \circ S$ of the successor function with itself.

Primitive recursion

Given $g : \mathbb{N}^k \to \mathbb{N}$ and $h : \mathbb{N}^{k+2} \to \mathbb{N}$, the function $f : \mathbb{N}^{k+1} \to \mathbb{N}$ is defined by *primitive recursion* from g and h if the following equalities hold:

$$\begin{array}{lcl} f(0, n_1, n_2, \ldots, n_k) & = & g(n_1, n_2, \ldots, n_k) \\ f(n{+}1, n_1, \ldots, n_k) & = & h(n, f(n, n_1, n_2, \ldots, n_k), n_1, \ldots, n_k) \end{array}$$

Here are some examples of primitive recursive definitions.

- The function $\mathit{plus}(n, m) = n{+}m$ can be defined by primitive recursion from $g = \Pi^1_1$ and $h = S \circ \Pi^3_2$. In other words,

$$\begin{array}{lcllcll} \mathit{plus}(0, n) & = & g(n) & = & \Pi^1_1(n) & = & n \\ \mathit{plus}(m{+}1, n) & = & h(m, \mathit{plus}(m, n), n) & = & S \circ \Pi^3_2(m, \mathit{plus}(m, n), n) & = & S(\mathit{plus}(m, n)) \end{array}$$

- The function $\mathit{times}(n, m) = n \cdot m$ can be defined by primitive recursion from $g = Z$ and $h = \mathit{plus} \circ (\Pi^3_3, \Pi^3_2)$.

Minimalization

Given $g : \mathbb{N}^{k+1} \to \mathbb{N}$, the function $f : \mathbb{N}^k \to \mathbb{N}$ is defined by *minimalization* from g if

$$f(n_1, n_2, \ldots, n_k) = \mu n.(g(n, n_1, n_2 \ldots, n_k) = 0)$$

where μ is the *minimalization operator*: $\mu n.P(n)$ returns the least natural number n such that $P(n)$ holds. If $P(n)$ does not hold for any n, then the result is undefined.

In modern algorithmic notation, f can be computed by a while loop of the form

```
n := 0;
while (g(n,n1,n2,...,nk) != 0) {n := n+1};
return n;
```

For example, consider the function $\log_2 n$ defined as follows: $\log_2 n = k$, if $2^k = n$. If n is not a power of 2, $\log_2 n$ is undefined. The function $\log_2$ can be defined by minimalization from the function $g(k, n) = 2^k - n$.[2]

[2]Technically, we have to be a bit more careful here. Since we are dealing only with functions over $\mathbb{N}$,

Recursive functions

The set of *recursive functions* is the smallest set which contains the initial functions and is closed with respect to composition, primitive recursion, and minimalization.

If we exclude minimalization from this construction, we get the set of *primitive recursive functions*, all of which are total functions. Minimalization is the only operator which introduces partial functions (or, from a computational standpoint, non-termination).

12.11 Encoding recursive functions in lambda calculus

Recall that we have already fixed the following definitions:

- $\langle n \rangle \equiv \lambda f x.(f^n x)$.
- $succ \equiv \lambda n f x.(f(nfx))$ such that $succ\langle n \rangle \rightarrow^* \langle n+1 \rangle$.

The function *succ* is the encoding $\langle S \rangle$ for the initial function *Successor*. We can define encodings for the other initial functions *Zero* and *Projection* as follows:

- $\langle Z \rangle \equiv \lambda x.(\lambda g y.y)$.
- $\langle \Pi_i^k \rangle \equiv \lambda x_1 x_2 \ldots x_k.x_i$.

Similarly, the composition of functions is easily defined in the lambda calculus. What remains is to encode primitive recursion and minimalization. We begin by defining encodings for constructing pairs and projecting out the components of a pair.

- $pair \equiv \lambda xyz.(zxy)$.
- $fst \equiv \lambda p.(p(\lambda xy.x))$.
- $snd \equiv \lambda p.(p(\lambda xy.y))$.

Observe that $fst(pair\ a\ b) \equiv (\lambda p.(p(\lambda xy.x)))((\lambda xyz.(zxy))\ a\ b) \rightarrow (\lambda p.(p(\lambda xy.x))) \rightarrow (\lambda z.(zab))(\lambda xy.x) \rightarrow (\lambda xy.x)ab \rightarrow (\lambda y.a)b \rightarrow a$. Similarly, we can show that $snd(pair\ a\ b) \rightarrow^* b$.

We now show how to encode primitive recursion. For notational simplicity, we restrict ourselves to the case where the function f being defined by primitive recursion has just one argument. Thus, the definition of primitive recursion becomes

$$\begin{array}{lcl} f(0) & = & g \\ f(n{+}1) & = & h(n, f(n)) \end{array}$$

$m-n$ would be defined to be 0 if $m < n$, so we have to modify the definition of $g(k,n)$ appropriately to take care of this.

Observe that g is just a constant—that is, some fixed number k. Inductively assume that h is a function defined by the lambda expression $\langle h \rangle$—in other words, for all numbers m and n, $\langle h \rangle \langle m \rangle \langle n \rangle \rightarrow^* \langle g(m,n) \rangle$. The aim is to come up with a lambda expression $\langle f \rangle$ defining f, such that for all n, $\langle f \rangle \langle n \rangle \rightarrow^* \langle f(n) \rangle$.

Consider the function $t(n) = (n, f(n))$. Then, t can be defined by primitive recursion as follows:

$$\begin{array}{lcll} t(0) & = & (0, f(0)) & = & (0, g) \\ t(n{+}1) & = & (n{+}1, f(n{+}1)) & = & (n{+}1, h(n, f(n))) \\ & & & = & (succ(fst(t(n))), h(fst(t(n)), sec(t(n)))) \end{array}$$

This is a simpler form of primitive recursion called *iteration*—think of it as repeatedly iterating a *step* function starting with $(0, g)$. (In programming language parlance, this is equivalent to translating a top-down recursive definition into an equivalent bottom-up iterative one).

The step function is lambda-definable by the following expression:

$$step \equiv \lambda x.pair(succ(fst\ x))(\langle h \rangle (fst\ x)(sec\ x)).$$

We can check that $step\ (pair\ \langle n \rangle \langle f(n) \rangle) \rightarrow^* pair\ \langle n{+}1 \rangle \langle f(n{+}1) \rangle$.

$$\begin{array}{rcl} step\ (pair\ \langle n \rangle\ \langle f(n) \rangle) & \rightarrow & pair\ \ (succ(fst(pair\ \langle n \rangle\ \langle f(n) \rangle)))\ (\langle h \rangle (fst(pair\ \langle n \rangle\ \langle f(n) \rangle))(sec(pair \langle n \rangle\ \langle f(n) \rangle))) \\ & \rightarrow & pair\ (succ\ \langle n \rangle)\ (\langle h \rangle \langle n \rangle \langle f(n) \rangle) \\ & \rightarrow & pair\ (succ\ \langle n \rangle)\ \langle h(n, f(n)) \rangle \\ & \rightarrow & pair\ \langle n{+}1 \rangle\ \langle h(n, f(n)) \rangle \\ & \rightarrow & pair\ \langle n{+}1 \rangle\ \langle f(n{+}1) \rangle. \end{array}$$

The encoding for t is as follows:

$$\langle t \rangle \equiv \lambda y.y\ step\ (pair\ \langle 0 \rangle\ \langle g \rangle)$$

We can check that for all n, $\langle t \rangle \langle n \rangle \rightarrow^* pair\ \langle n \rangle\ \langle f(n) \rangle$.

$$\begin{array}{rcl} \langle t \rangle \langle 0 \rangle & \rightarrow & \langle 0 \rangle\ step\ (pair\ \langle 0 \rangle\ \langle g \rangle) \rightarrow pair\ \langle 0 \rangle\ \langle g \rangle \\ \langle t \rangle \langle n{+}1 \rangle & \rightarrow & \langle n{+}1 \rangle\ step\ (pair\ \langle 0 \rangle\ \langle g \rangle) \\ & \rightarrow & step\ (\langle n \rangle\ step\ (pair\ \langle 0 \rangle\ \langle g \rangle)) \\ & \rightarrow & step\ (pair\ \langle n \rangle\ \langle f(n) \rangle) \text{(by ind. hyp.)} \\ & \rightarrow & pair\ \langle n{+}1 \rangle\ \langle f(n{+}1) \rangle \text{(by what has been proved above)} \end{array}$$

Since $f(n)$ is $sec(t(n))$, $\langle f \rangle$ is the following expression:

$$\langle f \rangle \equiv \lambda y.sec(\langle t \rangle y)$$

It is immediate that for all n, $\langle f \rangle \langle n \rangle \rightarrow^* \langle f(n) \rangle$.

Collapsing all the steps we have seen above, we can write down an expression PR that constructs a primitive recursive definition from the functions g and h:

$$PR \equiv \lambda hgy.sec(y(\lambda x.pair\ (succ\ (fst\ x))\ (h(fst\ x))\ (sec\ x)))(pair\ \langle 0\rangle\ g))$$

We could also have used recursive definitions of lambda experssions to encode primitive recursion. Recursive definitions can also be used to define minimalization. It turns out that every recursive definition in the lambda calculus can be "solved" by finding its fixed point.

12.12 Fixed points

We begin our discussion of fixed point by providing an encoding for boolean values true (`tt`) and false (`ff`) and conditional expressions. We begin with the conditional. We seek a term $\lambda bxy.\ E$ that will take three arguments, return x if b (the conditional expression) is true and y if b is false.

We claim that the expression $\lambda bxy.\ bxy$ with the definitions of `tt` and `ff` given by $\langle$`tt`$\rangle \equiv \lambda xy.x$ and $\langle$`ff`$\rangle \equiv \lambda xy.y$ do the job. (Incidentally, note that $\langle$`ff`$\rangle$ is the same as $\langle 0\rangle$).

Clearly

$$\begin{array}{rcl}(\lambda bxy.bxy)\langle\texttt{tt}\rangle fg & \rightarrow & \lambda xy.((\lambda xy.x)xy)fg \\ & \rightarrow & \lambda y.((\lambda xy.x)fy)g \\ & \rightarrow & (\lambda xy.x)fg \\ & \rightarrow & (\lambda y.f)g \\ & \rightarrow & f\end{array}$$

and, similarly,

$$\begin{array}{rcl}(\lambda bxy.bxy)\langle\texttt{ff}\rangle fg & \rightarrow & \lambda xy.((\lambda xy.y)xy)fg \\ & \rightarrow & \lambda y.((\lambda xy.y)fy)g \\ & \rightarrow & (\lambda xy.y)fg \\ & \rightarrow & (\lambda y.y)g \\ & \rightarrow & g\end{array}$$

Thus $\lambda bxy.bxy$ gives us an encoding of the construct `if-then-else`. We are now close to being able to write recursive function definitions in a syntax similar to languages like Haskell. For instance, we could aim to write

$$\begin{array}{rll}factorial\ \langle n\rangle = & \text{if } (iszero(\langle n\rangle)) & \text{then } 1 \\ & & \text{else } mult\ \langle n\rangle(factorial(pred\ \langle n\rangle))\end{array}$$

where, of course, we still have to encode the predicate *iszero* and the arithmetic function *pred* (predecessor).

Assuming we can do this, how do we convert such a recursive definition into a lambda term?

From recursive functional definitions to lambda terms

Let $F = \lambda x_1 x_2 \ldots x_n E$ be a recursive definition of F where E contains not only the variables x_1, x_2, ..., x_n and also the expression F. How do we transform such a definition into an equivalent lambda term?

To do this, we choose a new variable f and convert E to E^* by replacing every occurrence of F in E by (ff). That is, if E is of the form $\cdots F \cdots F \cdots$ then E^* is $\cdots (ff) \cdots (ff) \cdots$. Now, write

$$\begin{aligned} G &= \lambda f x_1 x_2 \ldots x_n . E^* \\ &= \lambda f x_1 x_2 \ldots x_n . \cdots (ff)\ cdots(ff) \cdots \end{aligned}$$

Then

$$GG = \lambda x_1 x_2 \ldots x_n . \cdots (GG) \cdots (GG) \cdots$$

This means that GG satisfies the defining equation for F. We can therefore write $F = GG$, where $G = \lambda f x_1 x_2 \ldots x_n . E^*$.

Encoding minimalization

Now that we know how to convert recursive definitions into lambda terms, we can encode minimalization as follows. Assume that we want to define a function $f : \mathbb{N}^k \to \mathbb{N}$ by minimalization from a function $g : \mathbb{N}^{k+1} \to \mathbb{N}$ for which we already have an encoding $\langle g \rangle$. We begin with a recursive definition of a term F as follows.

$$\begin{aligned} F = \lambda n x_1 x_2 \ldots x_k .\ \text{if } \mathit{iszero}(\langle g \rangle\ n\ x_1\ x_2\ \ldots\ x_k) \quad & \text{then } n \\ & \text{else } F(\mathit{succ}\ n)\ x_1\ x_2\ \ldots\ x_k \end{aligned}$$

Let $\tilde{F}$ be the lambda term corresponding to F after unravelling the recursive definition. The encoding $\langle f \rangle$ of f is then $\tilde{F}\ \langle 0 \rangle$.

All that remains is to provide an encoding for the predicate *iszero*.

Encoding of *iszero*

12.13 A fixed point combinator

Consider the recursive definition $F = \lambda x.x(Fx)$. By the previous "trick" for unravelling recursive definitions, we can find a lambda term for F as follows.

$$\begin{aligned} F &= GG, \text{where } G = \lambda f x.x(fffx) \\ &= \lambda f x.x(\lambda f x.x(fffx) \lambda f x.x(fffx) x) \end{aligned}$$

Notice that $FX = X(FX)$ for any lambda term X, by the definition of F. For any term Z, a fixed point of Z is a term M such that $ZM = M$. Clearly, if we set $M \equiv FZ$, we obtain a fixed point for Z. Notice that it does not matter what Z is—any lambda term Z has a fixed point FZ where F is the function we have just constructed. This fixed point operator is due to Turing and is traditionally denoted Θ.

12.14 Making sense of terms without normal forms

Recall that for terms with normal forms, the normal form denotes the "value" of the term. What about terms without normal forms? Are all terms without normal forms equally "meaningless"?

Suppose we want an equivalence $\approx$ on lambda terms such that:

1. $(\lambda x M)N \approx M\{x \leftarrow N\}$—that is, $\approx$ the equivalence induced by the β reduction.
2. If M and N do not have normal forms, then $M \equiv N$.
3. Functions that are equated by $\approx$ yield equivalent results for the same arguments. That is, if $M \approx N$ then for all R, $MR \approx NR$.

Then, we can show that $\approx$ is the trivial relation the equates all terms—in other words, $M \approx N$ for all M, N!

To see this, consider the function F defined by

$$Fxb = \text{ if } b \text{ then } x \text{ else } (Fxb)$$

As we have seen above, we can plug in our definition for `if-then-else` and then unravel this recursive definition to yield a lambda term for F.

It turns out that

$$F = GG, \text{ where } G = \lambda fxb.(\text{ if } b \text{ then } x \text{ else } (ffxb)$$

Now, consider $FX\langle \texttt{tt} \rangle$ and $FX\langle \texttt{ff} \rangle$, where $\langle \texttt{tt} \rangle$ and $\langle \texttt{ff} \rangle$ are the encodings of the truth values true and false.

- $FX\langle \texttt{tt} \rangle \rightarrow \text{if } \langle T \rangle \text{ then } X \text{ else } (FX\langle \texttt{tt} \rangle) \rightarrow X$.
- $FX\langle \texttt{ff} \rangle \rightarrow \text{if } \langle F \rangle \text{ then } X \text{ else } (FX\langle \texttt{ff} \rangle) \rightarrow FX\langle \texttt{ff} \rangle$.

Now, suppose we have an equivalence $\approx$ as described above. Then

$$\begin{array}{rcl} FZ & \rightarrow & (\lambda xb.(\text{if } b \text{ then } x \text{ else } (Fxb))Z \\ & \rightarrow & (\lambda b.(\text{if } b \text{ then } Z \text{ else } (FZb))) \\ & \rightarrow & (\lambda b.(\text{if } b \text{ then } Z \text{ else } G), \text{where } G = \text{if } b \text{ then } Z \text{ else } (fZB) \\ & \rightarrow & (\lambda b.(\text{if } b \text{ then } Z \text{ else (if } b \text{ then } Z \text{ else } G)) \\ & \rightarrow & \ldots \end{array}$$

Since FZ does not terminate for any Z, we have $FX \approx FY$ for all X and Y. But, since $FX \approx FY$ implies that $FXM \approx FYM$ for all M (by Condition 3 in the definition of $\approx$). But then we have $FX\langle \texttt{tt} \rangle \approx FY\langle \texttt{tt} \rangle$. Since $FZ\langle \texttt{tt} \rangle \rightarrow Z$ for all Z, this means that $X \approx Y$ for all X and Y!